attributes, attribute_names, and attribute_present? methods. The first returns a hash of attribute name-value pairs, the second returns an array of names, and the third returns true if a named attribute is available in this model object:

rails7/e1/ar/find_examples.rb

```
orders = Order.find_by_sql("select name, pay_type from orders")
first = orders[0]
p first.attributes
p first.attribute_names
p first.attribute_present?("address")
```

This code produces the following:

```
{"name"=>"Dave Thomas", "pay_type"=>"check"}
["name", "pay_type"]
false
```

find_by_sql can also be used to create model objects containing derived column data. If we use the as xxx SQL syntax to give derived columns a name in the result set, this name will be used as the name of the attribute:

rails7/e1/ar/find_examples.rb

```
items = LineItem.find_by_sql("select *,                                  " +
                             "  products.price as unit_price,            " +
                             "  quantity*products.price as total_price,  " +
                             "  products.title as title                  " +
                             " from line_items, products                 " +
                             " where line_items.product_id = products.id ")
li = items[0]
puts "#{li.title}: #{li.quantity}x#{li.unit_price} => #{li.total_price}"
```

As with conditions, we can also pass an array to find_by_sql, where the first element is a string containing placeholders. The rest of the array can be either a hash or a list of values to be substituted.

```
Order.find_by_sql(["select * from orders where amount > ?",
                   params[:amount]])
```

> **David says:**